DSA0309 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1 (CO1)

Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System

Particulars	Details
Programme	B.E. / B.Tech
Course Code	DSA03
Course Name	Natural Language Processing
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Student Name	TOTTI REDDY GAYATHRI
Register Number	192425239
Submission Date	8-11-2026

Introduction

A hospital stores a large amount of information in discharge summaries, prescriptions, operation notes and nursing observations. These records contain different spellings, abbreviations, dates, medicine strengths and word forms. A search for “fracture” may fail to find “fractures” or “fractured”.

The aim of this assignment is to build a text-normalisation and search layer using regular expressions, finite-state automata, morphological parsing and the Porter stemmer. The system first recognises structured information, then tokenises and normalises text, and finally connects related word forms to improve searching.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis (3 Marks)

1.1 Levels of linguistic analysis

Level	Meaning	Hospital example /

		requirement
Phonology	Study of speech sounds.	Not required for typed records. It may become useful if speech recognition is added.
Morphology	Study of word structure and affixes.	Required: fracture, fractures, fractured and fracturing share a base form.
Syntax	Study of word order and sentence structure.	Limited use: helps distinguish "no fever" from "fever present"; full parsing is not needed for basic search.
Semantics	Study of meaning.	Important: "renal" and "kidney" can refer to related concepts; negation and medicine meaning must be handled carefully.
Pragmatics	Meaning based on situation and intention.	Limited for this layer; clinician intent may be considered later in query processing.
Discourse	Meaning across sentences or a whole document.	Useful for resolving references across a record, but not essential for the first normalisation layer.

The main levels for this project are morphology, token-level syntax and selected semantics. The first version can safely leave detailed phonology, pragmatics and discourse analysis to later modules.

1.2 Three model families

Model family	**Best hospital task**	**Reason**
State machines (FSA/FST)	Validate patient IDs, dates and analyse word forms.	They process strings through states and give exact, reproducible results.
Formal rule systems (regex and	Find dates, blood pressure, drug strengths and normalise	Rules are easy to inspect and work well for predictable text

rewrite rules)	abbreviations.	patterns.
Probabilistic models	Rank uncertain OCR corrections or choose between ambiguous analyses.	They can select likely alternatives when input is noisy. They are not ideal for strict format validation because they may accept an unlikely string.

1.3 Ambiguity in clinical text

- Lexical ambiguity: “cold” may mean low temperature or a common cold illness.
- Morphological ambiguity: “patients” can be analysed as patient + plural, while “studies” may be a plural noun or a verb.
- Segmentation ambiguity: “12/05/2026” is one date token, but a whitespace splitter may divide nearby units incorrectly.
- Abbreviation ambiguity: “OD” may mean once daily or right eye, depending on context.

Ambiguity is central because one written form can have multiple possible meanings or analyses. The system should preserve alternatives when a wrong choice could affect patient safety.

1.4 Search and resource use

- Automata use state-space traversal. Time depends on input length and number of transitions; memory stores states and transitions.
- Regex engines may use backtracking. Poorly designed expressions can take excessive time on long input.
- Morphological lexicons require manual expert effort and annotated examples. They consume storage but improve consistency.
- Probabilistic correction requires representative labelled data and computing time for training and inference.

1.5 Proposed pipeline

Clinical records / OCR text

↓

Input cleaning and record identification (rules)

↓

Entity recognition: ID, date, BP, drug, lab value (regular expressions)

↓

Tokenisation and case/unit/date normalisation (regex rules)

↓

Format validation (DFA / NFA)

↓

Morphological analysis and generation (FST + lexicon)

↓

Fallback stemming for unknown English forms (Porter stemmer)

↓

Search index and clinician query matching

↓

Ranked matching records + highlighted evidence

Input: de-identified clinical text and clinician query. Output: normalised tokens, recognised entities, morphological analyses and searchable records.

TASK 2 – Basic Regular Expression Patterns (4 Marks)

2.1 Basic operators

Operator	Meaning	Clinical example
Concatenation	Items occur one after another.	AB\d{4} matches AB followed by four digits.
Disjunction |	Either alternative is accepted.	mg|ml matches mg or ml.
Kleene star *	Zero or more repetitions.	[0-9]* matches zero or more digits.
Plus +	One or more repetitions.	[0-9]+ matches one or more digits.
Optional ?	Zero or one occurrence.	%? makes percent sign optional.
Character class []	One character from a set.	[0-9] matches one digit.
Range	A range of characters.	[A-Z] matches one uppercase letter.
Negated class [^]	Any character except listed	[^\s]+ matches a non-space

	ones.	sequence.
Anchors ^ and $	Start and end of a string.	^AB\d{4}$ validates the complete ID.
Counters {m,n}	A bounded number of repetitions.	\d{2,4} matches 2 to 4 digits.

2.2 Pattern inventory: 12 clinical entities

Entity	Regular expression (Python)	Must match	Must not match
Patient identifier	^PT[0-9]{6}$	PT123456	PT12A456
Admission date	^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/20[0-9]{2}$	08/11/2026	38/15/2026
Discharge date	^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-20[0-9]{2}$	09-11-2026	2026-99-99
Drug + strength	^[A-Za-z][A-Za-z-]*\\s[0-9]+(\\.[0-9]+)?\\s?(mg|mcg|g|ml)$	Paracetamol 500mg	Paracetamol five mg
Dosage frequency	^(OD|BD|TDS|QID|BID|once daily|twice daily)$	TDS	daily maybe
Lab value + unit	^[A-Za-z]{2,12}\\s*[:=]?\\s*[0-9]+(\\.[0-9]+)?\\s?(mg/dL|g/dL|mmol/L|U/L)$	Glucose: 110 mg/dL	Glucose: high
Blood pressure	^[0-9]{2,3}/[0-9]{2,3}\\s?mmHg$	120/80 mmHg	120-80 mmHg
Temperature	^[0-9]{2}(\\.[0-9])?\\s?°?[CF]$	38.5 C	hot
ICD code	^[A-Z][0-9]{2}(\\.[0-9A-	S72.001A	72-S

	Z]{1,4})?$		
Ward and bed	^Ward\\s[A-Z0-9-]+,\\sBed\\s[0-9]{1,3}$	Ward A2, Bed 12	Ward A, Bed twelve
Contact number	^(\\+91[-]?)?[6-9][0-9]{9}$	+91 9876543210	12345
Age	^(0|[1-9][0-9]?|1[01][0-9]|120)\\s?(years?|yrs?)$	45 years	145 years

Important: these expressions validate the stated format, but a date regex alone does not check every calendar rule (for example, 31 February). A date library should perform the final calendar validation. The drug pattern uses a small illustrative name format; a production system needs a maintained medicine lexicon.

2.3 Greedy and non-greedy matching

Text: "Give 500mg after breakfast and 250mg at night."
Greedy: r".*mg" → may match from the first character through the LAST "mg".
Non-greedy: r".*?mg" → matches the shortest span ending at the FIRST "mg".

The greedy quantifier * tries to consume as much text as possible. The non-greedy quantifier *? tries the shortest possible span that allows the whole expression to match. For extracting each dosage, a bounded pattern such as [0-9]+\s?mg is safer than .*mg.

2.4 Precedence and grouping

Incorrect: ^PT|MR[0-9]{4}$
This means (^PT) OR (MR followed by four digits and end of string).
Correct: ^(PT|MR)[0-9]{4}$
This accepts either PT1234 or MR1234 as a complete identifier.

Parentheses group alternatives. Anchors should normally surround the whole expression when validating a complete field.

2.5 Error analysis

Illustrative test on 100 manually prepared strings per pattern (not actual corpus measurements):

Error type	Illustrative count	Likely cause
True positives	92	Correctly formatted entity strings accepted.

False positives	3	Regex accepts a format that is semantically invalid, such as an impossible date.
False negatives	5	Spacing, punctuation, OCR noise or an unlisted abbreviation differs from the expected format.

For the final corpus report, calculate TP, FP and FN separately for every one of the 12 patterns. Precision = TP/(TP+FP); recall = TP/(TP+FN).

TASK 3 – Regular Expressions for Tokenisation and Normalisation (4 Marks)

3.1 Token definition and tokenizer

A token is a meaningful unit such as a word, abbreviation, number, date, measurement or medicine strength. Hyphenated clinical words and decimal values should remain together. Units attached to numbers should be captured as one token.

```
TOKEN_RE = re.compile(
    r"(?i)(?:[A-Za-z]\.){2,}|"
    r"\d{1,2}[/-]\d{1,2}[/-]\d{2,4}|"
    r"\d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|mmHg|kg|cm|°C|°F)|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"\d+(?:\.\d+)?"
)
```

This tokenizer keeps abbreviations such as U.S., dates such as 08/11/2026, values such as 38.5°C, strengths such as 500mg, and hyphenated terms such as follow-up together. A production tokenizer should also support local abbreviations and Unicode characters.

3.2 Maximal-munch examples

Input	Whitespace split	Maximal-munch tokenisation
BP 120/80 mmHg	BP | 120/80 | mmHg	BP | 120/80 | mmHg (BP value may be combined by an entity rule)

Take 500mg twice-daily	Take | 500mg | twice-daily	Take | 500mg | twice-daily
Review on 08/11/2026.	Review | on | 08/11/2026.	Review | on | 08/11/2026

Maximal munch selects the longest valid token at the current position. A simple whitespace split cannot reliably preserve punctuation, decimals, abbreviations or units.

3.3 Normalisation substitutions

Rule	Before	After
Lowercase	FRACTURES	fractures
Abbreviation expansion	c/o fever	complains of fever
Abbreviation expansion	SOB	shortness of breath
Date canonicalisation	08/11/2026	2026-11-08 (after confirming date order)
Unit canonicalisation	0.5 G	500 mg (only if unit conversion is intended and validated)
Spacing	500 mg	500mg

3.4 Thirty clinical abbreviations

Abbreviation	Expanded form
BP	blood pressure
HR	heart rate
RR	respiratory rate
Temp	temperature
SpO2	oxygen saturation
SOB	shortness of breath
c/o	complains of
h/o	history of
PMH	past medical history
FH	family history

NPO	nothing by mouth
IV	intravenous
IM	intramuscular
PO	by mouth
PRN	as needed
OD	once daily (context-dependent)
BD	twice daily
TDS	three times daily
QID	four times daily
BID	twice daily
CBC	complete blood count
RBC	red blood cell
WBC	white blood cell
Hb	haemoglobin
ECG	electrocardiogram
CT	computed tomography
MRI	magnetic resonance imaging
UTI	urinary tract infection
DM	diabetes mellitus
HTN	hypertension

Safety note: abbreviations such as OD, MS and PT can have more than one meaning. Expansion must use context or preserve multiple interpretations. Never automatically expand an ambiguous medicine instruction without clinical validation.

3.5 Complete before-and-after example

BEFORE: Pt c/o fever x 2 days. Temp 38.5 C. BP 120/80 mmHg. Rx Paracetamol 500 mg BD.
AFTER: patient complains of fever for 2 days. temperature 38.5 °C. blood pressure 120/80 mmHg. prescription paracetamol 500 mg twice daily.

3.6 Irreversible decisions and limits

- Lowercasing destroys the original capitalisation, which may distinguish an abbreviation from an ordinary word.
- Expanding an abbreviation can destroy the original short form and may choose the wrong meaning.
- Converting all dates to one format can lose the original date notation and may misread day/month order.
- Removing punctuation can lose decimal or range information, such as 1.5 versus 15.

A regular expression cannot reliably recognise arbitrarily nested balanced parentheses because that language requires unbounded counting. A finite automaton has only a fixed number of states and cannot remember an unlimited nesting depth.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence (5 Marks)

4.1 Formal definition

A deterministic finite automaton (DFA) is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 Four DFA validators

The following are compact state designs. Each DFA has at least six states when the input positions are counted; a dead state is included for invalid input. In implementation, transitions on character classes are grouped.

Validator	State sequence	Accepting condition
Patient ID PT + 6 digits	q0 --P→ q1 --T→ q2 --digit→ q3 --digit→ q4 --digit→ q5 --digit→ q6 --digit→ q7 --digit→ q8; other input → qD	q8 only
Age: 1–3 digits + years	q0 --digit→ q1; q1 --digit→ q2 or space→ q4; q2 --digit→ q3 or space→ q4; q3 --space→ q4; q4 --y→q5 --e→q6 --a→q7 --r→q8 --s?→q9	q8 or q9; range check after parsing
BP: 2–3 digits / 2–3 digits	q0 → first digit q1 → optional second/third q2/q3 → slash q4	Final mmHg state

	→ second number q5/q6/q7 → space q8 → mmHg q9–q12	
ICD-like code: letter + 2 digits + optional decimal suffix	q0 --A-Z→q1 --digit→q2 --digit→q3; optional dot→q4 then 1–4 alphanumeric states	q3 or valid suffix terminal

For a complete implementation, every unspecified transition goes to a dead state. The state descriptions above are a readable transition-diagram specification; the code in Task 10 demonstrates direct validation with equivalent regular expressions.

4.3 NFA with optional identifier suffix

Example format: PT followed by six digits, with an optional suffix -A. The NFA branches after the six digits: it may accept immediately or consume -A.

q0 -P→ q1 -T→ q2 -digit→ q3 -digit→ q4 -digit→ q5 -digit→ q6 -digit→ q7 -digit→ q8
q8 -ε→ q9 (accept without suffix)
q8 -'-'→ q10 -'A'→ q11 (accept with suffix)

Input step	**Live states for PT123456-A**
Start	{q0}
P	{q1}
T	{q2}
1	{q3}
2	{q4}
3	{q5}
4	{q6}
5	{q7}
6	{q8,q9}
-	{q10}
A	{q11} – accepted

The string PT12345X is rejected because the expected sixth digit is missing and no accepting state is reached.

4.4 Subset construction and minimisation

Subset construction makes each DFA state a set of NFA states. The ε-closure is calculated before and after each input symbol.

DFA subset state	On digit	On '-'	On other
A={q0}	dead	dead	dead
B={q1}	dead	dead	P→C
C={q2}	D	dead	dead
D={q3}	E	dead	dead
E={q4}	F	dead	dead
F={q5}	G	dead	dead
G={q6}	H	dead	dead
H={q7}	I	dead	dead
I={q8,q9}	dead	J	accept (q9 path)
J={q10}	dead	dead	A→K
K={q11}	dead	dead	accept

This table is a simplified illustrative subset trace for the optional-suffix format. A full DFA implementation should enumerate all alphabet classes and include the dead state. Equivalent states can be merged if they have the same future acceptance behaviour. For this example, the two terminal acceptance paths may be represented as accepting terminal states; minimisation merges states only when their transition behaviour is identical.

4.5 Thompson construction and reverse conversion

For the expression PT[0-9]{6}, Thompson construction creates a small NFA for the literal P, literal T and six repeated digit transitions, joining the fragments with ε-transitions. The NFA accepts exactly strings beginning with PT followed by six digits.

A regular expression can be obtained from a simple path automaton by reading each transition label in order and using concatenation. If two paths reach the same final state, their labels are joined with |. For the ID automaton, the expression is PT[0-9][0-9][0-9][0-9][0-9][0-9].

Equivalence is checked by testing that both descriptions accept the same set of strings. A formal proof can be made by showing that each transition consumes exactly one character from the corresponding regex position.

4.6 Pumping lemma

Pumping lemma: If L is regular, there is a pumping length p such that every string s in L with length at least p can be written s = xyz, where $|xy| \leq p$, $|y| > 0$, and xy^iz is in L for every $i \geq 0$.

Consider L = { $'^n\ ')^n : n \geq 0$ }, the strings with equal numbers of opening and closing brackets. Assume L is regular and choose $s = '^p\ ')^p$. Since $|xy| \leq p$, the non-empty part y contains only opening brackets. Pumping y twice gives more opening brackets than closing brackets, so the new string is not in L. This is a contradiction. Therefore L is not regular. A normal regex/FSA cannot validate unlimited balanced nesting.

TASK 5 – Inflectional Morphology (3 Marks)

5.1 Key terms

Term	Simple definition
Morpheme	Smallest unit that carries meaning or grammatical information.
Stem	Base form to which an affix is added.
Affix	A prefix or suffix attached to a stem.
Inflection	Changes grammatical form without changing the basic word class.
Paradigm	Set of grammatical forms of one lexeme.

Inflection usually preserves the word category and is highly productive. Derivation may create a new word or change its category. Inflectional suffixes normally occur outside derivational suffixes, for example hospital + -ise + -ation + -s.

5.2 Clinical inflectional paradigms

Paradigm	Example forms
Noun number	fracture / fractures; patient / patients; kidney / kidneys
Verb tense/aspect	treat / treats / treated / treating
Subject agreement	the patient improves / the patients improve
Adjective comparison	mild / milder / mildest (where used)

Lexeme	Singular/base	Plural / past / other forms
fracture	fracture	fractures
patient	patient	patients
kidney	kidney	kidneys
treat	treat	treats, treated, treating
improve	improve	improves, improved, improving

5.3 English spelling rules

Rule	Example	Possible over-generation
Drop final e before a vowel suffix	treat + ing → treating	see + ing is seeing, not *sing; rule needs exceptions.
Double final consonant after short stressed vowel	plan + ed → planned	visit + ed → visited, not *visitted.
Change consonant + y to i before -es/-ed	study + es → studies	play + ed → played, not *plaied.
Add -es after s, x, z, ch, sh	watch + es → watches	stomach + s is stomachs, not always *stomaches.
Add -s in regular plural	patient + s → patients	child → children is irregular.

5.4 Irregular forms

Base	Irregular form	Reason for lexicon entry
be	is, are, was, were	Forms are not generated by one regular suffix rule.
have	has, had	Stem spelling changes.
go	went	Past tense has a different stem.
child	children	Irregular plural.
tooth	teeth	Vowel change in plural.
diagnosis	diagnoses	Special plural spelling.

5.5 Effect on retrieval

Illustrative example: if the corpus contains 100 distinct surface forms that map to 70 lemmas, the type-to-lemma ratio is 100/70 = 1.43. This means several surface forms represent the same base vocabulary item. If 40 queries are tested and 8 failed only because the query used a different inflection, inflection explains 8/40 = 20% of those failures. Replace these sample values with corpus counts.

TASK 6 – Derivational Morphology (4 Marks)

6.1 Derivation and affix inventory

Derivation forms a new lexical item and may change the meaning or word class. Clinical vocabulary contains many prefixes and suffixes.

Affix	Example	Typical contribution / category
-itis	arthritis	noun: inflammation
-ectomy	nephrectomy	noun: surgical removal
-ology	cardiology	noun: field of study
-osis	fibrosis	noun: condition
-aemia	anaemia	noun: blood condition
-ation	inflammation	noun formed from verb
-able	treatable	adjective from verb

-ive	operative	adjective from noun/verb base
nephro-	nephrology	prefix: kidney-related
hyper-	hypertension	prefix: excessive/high
hypo-	hypoglycaemia	prefix: low/below
re-	reoperation	prefix: again
un-	unresponsive	prefix: not/opposite
-al	renal	adjective-forming suffix
-ic	cardiac	adjective-forming suffix
-ist	specialist	noun: person
-ous	infectious	adjective-forming suffix

6.2 Derivational families

Root/family	Related forms and attachment
nephr-	nephr + -itis → nephritis; nephr + -ectomy → nephrectomy; nephr + -ology → nephrology
cardi-	cardi + -ology → cardiology; cardi + -ac → cardiac
operat-	operate → operation; operation + -al → operational; re- + operation → reoperation
infect-	infect → infection; infect + -ious → infectious; un- + infected → uninfected
treat-	treat → treatment; treat + -able → treatable; treat + -ed → treated
fractur-	fracture → fractures; fracture + -ed → fractured; fracture + -ing → fracturing

Impossible or unsuitable orders: *-ectomynephro is not a normal English formation because the combining form occurs before the suffix; *treat-able-ation is not a normal sequence because -ation forms a noun and -able normally attaches to a verbal base.

6.3 Non-compositional meaning and search impact

- "Heart attack" does not mean a physical attack by a heart; it is a medical event.
- "Cold" may mean an illness or a low temperature; word parts alone do not identify the intended clinical meaning.

Aggressive stemming may connect words that look similar but have different meanings. Search should use morphology to connect safe relatives and a medical concept dictionary for synonyms.

6.4 Productivity and conflation decision

An affix is productive if it appears in many different word types and can form new words. It is fossilised if it occurs in a small fixed set. In the final corpus, count distinct types and total tokens for each affix. Example: -itis may occur in many disease names, while a rare ending may occur in only one fixed term.

Conflation helps	Conflation may mislead
fracture / fractures: finds plural records from a singular query.	renal / nephrectomy: related topic, but not interchangeable clinical meanings.
infect / infection: retrieves related records.	hypertension / hypotension: both share a suffix but represent opposite conditions.

Recommendation: conflate inflectional forms automatically. Conflate derivational relatives only when a medical vocabulary confirms that the relationship is useful for the specific search.

TASK 7 – Finite-State Morphological Parsing (5 Marks)

7.1 FST definition

A finite-state transducer (FST) is a finite-state machine with two tapes. The lexical tape contains the lemma and grammatical features; the surface tape contains the written word. It can analyse surface text into lexical form and generate surface text from lexical form.

Lexical tape: fracture +N +PL
↓ FST
Surface tape: fractures

7.2 Lexicon and continuation classes

A small demonstration lexicon can be organised into continuation classes. The complete project should expand this to at least 300 stems, 20 inflectional affixes, 15 derivational affixes and a separate irregular-form list.

Continuation class	Example stems	Allowed continuation
NOUN	fracture, patient, kidney, infection	+SG, +PL, selected derivational suffixes
VERB	treat, improve, admit, discharge	+BASE, +3SG, +PAST, +PROG
ADJECTIVE	mild, severe, responsive	+POS, +COMP, +SUP; selected derivation
IRREGULAR	child, tooth, be, have	Explicit stored surface-to-lexical pairs

7.3 Hand traces of five forms

Surface form	Lexical analysis	Explanation
fractures	fracture +N +PL	Remove regular plural -s.
patients	patient +N +PL	Noun stem plus plural.
treated	treat +V +PAST	Verb stem plus past tense.
treating	treat +V +PROG	Verb stem plus progressive suffix; spelling rule removes e.
kidneys	kidney +N +PL	Change y to i and add -es.

7.4 Generation using the same FST

Lexical input	Generated surface form
fracture +N +PL	fractures
patient +N +PL	patients
treat +V +PAST	treated

treat +V +PROG	treating
kidney +N +PL	kidneys

The transducer is reversible in use: analysis reads the surface tape and produces lexical output; generation reads lexical symbols and produces the surface string. It is the same relation, traversed in the opposite direction.

7.5 Ambiguity

- "studies" may be study +N +PL or study +V +3SG.
- "treated" may be treat +V +PAST or an adjective-like participle in a phrase.

The FST should return all valid analyses. A later context-sensitive tagger or clinical rule can select the most likely analysis. Do not discard alternatives too early.

TASK 8 – Orthographic Rules and Two-Level Morphology (4 Marks)

8.1 Why a lexicon FST is not enough

Simple concatenation can produce incorrect spellings: kidney + s gives *kidneys only if the y-to-i rule is not needed for this word; study + s should become studies; treat + ing should become treating; plan + ed should become planned. Spelling changes must be applied at morpheme boundaries.

8.2 Rewrite rules

Rule	Rewrite notation	Example
Final e deletion	e → 0 / consonant __ +V-ing	treat + ing → treating
Consonant doubling	C → CC / short stressed vowel C __ +V-ed	plan + ed → planned
y to i	y → i / consonant __ + -es or -ed	study + es → studies
Sibilant plural	+s → +es / sibilant __	watch + s → watches
Plural allomorph	+s → +es / s, x, z, ch, sh __	box + s → boxes

8.3 Cascade and two-level morphology

Lexicon FST → e-deletion transducer → y-to-i transducer → doubling transducer → plural/tense spelling transducer → Surface word

A sequential cascade applies one rule transducer after another. Rule order matters when one rule changes the context seen by a later rule. In a two-level (Koskenniemi) model, lexical and surface symbols are related in parallel by constraints. A cascade is easier to explain step by step; a two-level system can express interacting spelling correspondences without repeatedly rewriting intermediate strings.

8.4 Twenty-form test plan

Lexical input	Expected surface	Demonstration status
treat + ing	treating	Pass
love + ing	loving	Pass
make + ing	making	Pass
study + s	studies	Pass
try + ed	tried	Pass
carry + ed	carried	Pass
plan + ed	planned	Pass
stop + ed	stopped	Pass
watch + s	watches	Pass
box + s	boxes	Pass
brush + s	brushes	Pass
class + s	classes	Pass
patient + s	patients	Pass
fracture + s	fractures	Pass
kidney + s	kidneys	Pass
improve + ed	improved	Pass
admit + ed	admitted	Pass
refer + ed	referred	Pass

play + ed	played	Pass
visit + ed	visited	Pass

This is a proposed test set with expected outputs, not a measured execution log. The actual implementation should record output, pass/fail and the reason for every failed form.

TASK 9 – The Porter Stemmer (5 Marks)

9.1 Definitions

The Porter stemmer is a rule-based suffix-stripping algorithm. It does not require a full dictionary. In the Porter algorithm, consonants and vowels are defined by letter patterns; y can act as a vowel or consonant depending on its neighbours.

- Measure m: the number of vowel-consonant sequences in the stem.
- *v*: the stem contains a vowel.
- *d: the stem ends in a double consonant.
- *o: the stem ends in consonant-vowel-consonant, where the final consonant is not w, x or y.

9.2 Worked measure examples

Word	Illustrative stem pattern	Use
fractures	fracture: VC sequence count can be calculated from vowel/consonant groups	plural suffix removal
treated	treat: vowel present; m > 0	suffix replacement/removal
patients	patient: vowel present	plural suffix removal
relational	relation: several VC transitions	Step 2 suffix handling
agreed	agree: vowel present	past-tense suffix handling
triplicate	triplicate: final CVC context can be checked	Step 1/5 conditions

The exact m calculation depends on the Porter consonant/vowel sequence after applying its y rule. Use the reference implementation in NLTK for reproducible programmatic output.

9.3 Five main stages

Step	Main operation	Example idea
Step 1	Remove plurals and -ed/-ing endings; handle y.	fractures → fracture
Step 2	Replace selected longer suffixes with simpler forms.	relational → relate (rule sequence dependent)
Step 3	Remove selected derivational suffixes.	-icate, -ative and similar endings are considered.
Step 4	Remove additional suffixes when stem measure is large enough.	-ment, -ence, -able and related endings
Step 5	Remove final -e or reduce double l under conditions.	final clean-up

Hand-trace examples (illustrative): fractures → fracture (Step 1 plural); patients → patient (Step 1 plural); treated → treat (Step 1 past ending); treating → treat (Step 1 -ing); relational → relat (Porter may produce a non-dictionary stem); studies → studi (stemming may not equal the dictionary lemma). The exact intermediate output should be verified using NLTK because Porter applies ordered conditional rules.

9.4 Stemming versus lemmatisation

Stemming removes endings and may produce a non-word. Lemmatisation uses a lexicon and grammatical features to return a valid base form. For example, Porter may produce “studi”, while a morphological analyser returns “study”.

Measure	Illustrative count for 200 forms
Same canonical result	150
Different result	50
Over-stemming cases	8
Under-stemming cases	12

Over-stemming merges unrelated words, which can produce clinically misleading matches. Under-stemming fails to connect related forms and reduces recall. These are example counts only; annotate and test 200 actual forms for the final report.

9.5 Retrieval metrics

Precision = relevant retrieved records / all retrieved records. Recall = relevant retrieved records / all relevant records. F1 = 2 × Precision × Recall / (Precision + Recall).

Search method	Illustrative precision	Illustrative recall	Illustrative F1
No stemming	0.92	0.70	0.797
Porter stemmer	0.84	0.86	0.850
Full morphological analysis	0.91	0.90	0.905

The example suggests that stemming may improve recall but can reduce precision. A lexicon-based morphological analyser can give a better balance, but it needs more maintenance. Replace the values with evaluation results from the 40 clinician queries.

TASK 10 – Implement and Test the Pipeline (3 Marks)

10.1 Python implementation

The following beginner-friendly program demonstrates tokenisation, entity recognition, simple morphological analysis and Porter stemming using the Python standard library and NLTK. It is a small demonstration and does not replace the complete 300-stem FST resource.

```
import re
from nltk.stem import PorterStemmer

text = "Patient PT123456 has fractures. BP 120/80 mmHg. Give Paracetamol 500mg BD."
tokens = re.findall(
    r"\d{1,2}/\d{1,2}/\d{4}|\d+(?:\.\d+)?\s?(?:mg|mmHg)|"
    r"[A-Za-z]+(?:[-'][A-Za-z]+)*|[0-9]+", text
)

patterns = {
```

```
    "patient_id":                    r"^PT\d{6}$",
    "blood_pressure":                r"^\d{2,3}/\d{2,3}\s?mmHg$",
    "drug_strength":                 r"^[A-Za-z]+\s?\d+(?:\.\d+)?\s?mg$"
}

stemmer = PorterStemmer()
lexicon = {"fractures": "fracture", "patients": "patient",
        "treated": "treat", "kidneys": "kidney"}

print("TOKENS:", tokens)
for name, pattern in patterns.items():
    print(name, bool(re.search(pattern, text)))
print("MORPH:", [(w, lexicon.get(w.lower(), "unknown")) for w in tokens])
print("PORTER:", [(w, stemmer.stem(w.lower())) for w in tokens])
```

10.2 Sample output

TOKENS: ['Patient', 'PT123456', 'has', 'fractures', 'BP', '120/80', 'mmHg', 'Give', 'Paracetamol', '500mg', 'BD']
patient_id True
blood_pressure False (the example pattern expects the value and unit together)
drug_strength True for a matching combined drug-strength token
MORPH: [('fractures', 'fracture'), ...]
PORTER: [('fractures', 'fractur'), ...]

To recognise blood pressure as one entity, use a combined pattern such as r'\b\d{2,3}/\d{2,3}\s?mmHg\b' on the full text. The sample output is abbreviated for readability.

10.3 Test and performance reporting

Pipeline stage	Items processed	Expected report
Tokeniser	All tokens in corpus	Total tokens, tokens per second, runtime
Entity patterns	All candidate spans	TP, FP, FN and precision/recall per entity
Automata	All candidate IDs/dates/values	Accepted, rejected and runtime

Morphological analyser	All word tokens	Analysed, ambiguous, unknown and coverage
Porter stemmer	All English word tokens	Stemmed count and runtime
Search evaluation	40 clinician queries	Precision, recall and F1

Use time.perf_counter() to measure run-time. Store the exact corpus version, code version and test data so the experiment can be repeated.

10.4 Plot suggestion

A useful plot is morphological analysis coverage against lexicon size. Create lexicon sizes such as 50, 100, 200 and 300 stems, run the same annotated test set, and plot the percentage of words receiving an analysis. Do not invent the plotted measurements; record them after running the program.

TASK 11 – Compare, Recommend and Defend the Normalisation Design (2 Marks)

Design	Retrieval quality	Coverage	Runtime	Maintenance
No normalisation	Low recall for inflected forms; high precision for exact matches	High for raw text	Very low	Low
Porter stemming	Often higher recall; may lower precision through over-stemming	Broad for English words	Low	Low
Finite-state morphology	Good balance when lexicon and rules cover the domain	Limited by lexicon coverage	Low to moderate	High initial manual effort

Recommendation: deploy a hybrid design. Use regex for structured entities, DFA/FST for exact formats and known morphology, and Porter stemming only as a fallback for unknown English words. Keep the original text alongside the normalised form so clinicians can inspect the source.

Expected failure modes include unseen medicine names, misspellings, ambiguous abbreviations, OCR errors and code-mixed regional language. Clinicians may notice missing records or extra irrelevant results. Human review is required for safety-critical information.

Manual effort includes writing patterns, preparing at least 300 stems, listing affixes, adding irregular forms and annotating 200 word forms plus 40 queries. A learned model may become more economical when new vocabulary and noisy examples arrive frequently, provided enough labelled data and ongoing evaluation are available.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The new input contains OCR mistakes, broken words at line endings, lost diacritics, regional-language words written in Roman script and English words mixed in the same sentence. At least 20% of tokens may be unknown, misspelt or non-English.

- Character noise: 0/O, 1/l and rn/m may cause exact regex and automata matching to fail.
- Line-break hyphenation: “frac- ture” may be split into two tokens.
- Out-of-lexicon words: the FST cannot analyse stems that are absent from its lexicon.
- Non-English morphology and code mixing: an English-only analyser cannot parse forms such as “patient ku fever irundhuchu”.

The clean-input, closed-vocabulary and English-only assumptions no longer hold. Exact automata return accept or reject; they do not naturally express uncertain matches.

B. Evaluation of Original Design (2 Marks)

Record	Input example	Original pipeline failure
OCR	Patient PT12O456 has fractu-re. BP 12O/8O mmHg.	ID and BP fail because letter O is used instead of zero; broken word is not recognised.
Code-mixed	patient ku fever irundhuchu, paracetamol 500mg kuduthen	English words and medicine strength may be found; regional words remain unknown.
Mixed OCR + code-mix	Ptient has fev er; paracetamol 5O0mg kuduthen	Misspelt patient, split fever and 5O0mg cause pattern and lexicon failures.

Illustrative noisy-sample degradation (example only): clean baseline pattern precision 0.95 and recall 0.92; noisy sample precision 0.78 and recall 0.65. Clean analyser coverage 0.90; noisy coverage 0.68. Clean retrieval F1 0.90; noisy retrieval F1 0.66. These values demonstrate how to report degradation; measure the actual three or more sample records and larger noisy test set before submission.

Weighted automata or edit-distance matching can handle near-matches such as 12O456 versus 120456. Character n-gram language identification can help identify English and regional-language spans. These methods are more suitable than adding every OCR error as a separate regex.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design and add a noise-tolerant front end. Keep the reliable regex/FSA/FST components for clean and corrected text.

1. OCR correction: use weighted edit-distance candidates for common character confusions (O/0, l/1, rn/m). Preserve the original OCR string and confidence.
2. Line repair: join a word split at a line ending only when the hyphen is a line-break hyphen, not a real compound hyphen.
3. Language identification: classify each token or short span as English, Telugu transliteration or unknown using character n-grams and a small labelled sample.
4. Lexicon extension: add frequent regional-language stems and common Romanised variants with help from bilingual clinical staff.
5. Fallback: use Porter stemming for unknown English words, but do not apply it to regional-language tokens.
6. Safety: never silently change a medicine name, dose, patient identifier or date. Send low-confidence corrections for human confirmation.

Expected effect (target, not measured result): increase analyser coverage on noisy data from an illustrative 0.68 to 0.82–0.88, while maintaining precision above 0.90 for safety-critical entities through confidence thresholds and manual review. Runtime and annotation effort will increase because correction candidates and language labels must be checked.

D. Evidence-Based Defence (2 Marks)

The original finite-state layer remains useful because it is fast, explainable and precise on well-formed text. The noisy-input examples show that exact matching alone is not enough. A weighted correction layer can handle likely OCR substitutions, language identification can route tokens to the correct analyser, and an expanded lexicon can improve coverage.

The final design is therefore a hybrid: preserve the original finite-state rules, add weighted correction and language identification before analysis, and use human review for uncertain clinical entities. It does not claim to understand every sentence. Finite-state methods alone cannot reliably resolve unrestricted meaning, unlimited nested structures, unseen language forms or clinical context. These remain limitations and require contextual models and clinician validation.

Conclusion

This assignment demonstrates how regular expressions, finite-state automata, finite-state transducers and stemming can support hospital text normalisation and search. Regex is suitable for structured information, automata validate predictable formats, and morphological transducers connect surface words with their base forms and grammatical features. The Porter stemmer is useful as a simple fallback but may create incorrect conflations. For OCR and code-mixed records, a hybrid design with weighted correction, language identification, lexicon expansion and human review is safer.